

IA Analysis User Manual

Structured usage guide for the `ia_analysis` Python package

Version 0.1.0 - generated 2026-06-21

This manual explains how to install, navigate, run, and extend the project. It is written for analysis notebooks, local development, and COSMA/TNG production runs.

1. Project Overview

ia_analysis is a source-first Python package for intrinsic-alignment, shape-tide, halo-dynamics, real-space correlation, and power-spectrum analysis on ClusterSims and IllustrisTNG-style data.

- Use import paths under ia_analysis.* instead of root-level Python wrappers.
- Keep data paths outside source files and pass them through CLI flags, environment variables, or configuration files.
- Use structured API facades for notebooks and scripts; they lazy-load heavier implementations only when needed.
- Use notebooks for orchestration and exploration, but move reusable code into src/ia_analysis modules.

2. Repository Layout

Path	Purpose
configs/	Example path and environment configuration, including COSMA/TNG conventions.
docs/	Architecture notes, package guides, and this PDF manual.
notebooks/pipelines/	Maintained workflow notebooks with curated Markdown notes.
requirements/	Core, development, and COSMA dependency lists.
src/ia_analysis/	Importable Python package source.
tests/	Synthetic smoke tests and structural tests.
tools/	Maintenance helpers for notebook export and project management.

3. Dependency Direction

The package is split by scientific responsibility. Lower layers should not import high-level orchestration code.

```
catalogs / shapes / tides
    -> dynamics / MergerTree
    -> correlations / spectra / covariance
    -> pipelines
```

The visualization and orbits packages are side modules used by notebooks and experiments. Heavy optional dependencies stay inside the functional subpackages that need them.

4. Installation

Recommended local installation:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e .
python -m pip install -r requirements/dev.txt
```

Quick local import without installation:

```
set PYTHONPATH=src
python -c "import ia_analysis; print(ia_analysis.__version__)"
```

Baseline dependencies include numpy, scipy, h5py, numba, matplotlib, seaborn, pandas, tqdm, and psutil. h5py is a required baseline dependency because the project reads and writes HDF5 catalogs, snapshots, spectra, covariance products, and correlation outputs.

COSMA-specific dependencies such as Pylians, illustris_python, halotools, and pycccl may require local modules or separate installation. See requirements/cosma.txt.

5. Configuration And Data Paths

Do not hard-code local data paths inside source modules. Prefer CLI flags, environment variables, or configs/example_paths.json.

- TNG_API_KEY: optional API key for TNG downloads.
- TNG_SIM_NAME: default simulation name such as TNG300-1.
- TNG_CACHE_DIR: local cache directory for downloaded TNG products.
- ClusterSims and COSMA paths should be passed into pipelines or catalog loaders explicitly.

6. Subpackage Map

Subpackage	Main responsibility
catalogs	ClusterSims and TNG catalog, subhalo, FoF, snapshot, and particle loading.
shapes	Inertia tensors, axes, spin, kappa rotation, shape errors, and IA projection helpers.
tides	Potential grids, CIC gridding, tidal tensors, and interpolation.
dynamics	Shell-wise halo dynamics, affine flow, figure rotation, TNG wrappers, and binding-energy profiles.
MergerTree	Cross-snapshot TNG tree reading and workflow orchestration.
correlations	Real-space density, IA, velocity, omega, covariance, and compressed four-point correlations.
spectra	Mesh construction, IA/matter/velocity spectra, folded spectra, and NLA theory.
covariance	Gaussian, cNG, and SSC covariance products for spectra.
pipelines	End-to-end CS/TNG command entrypoints.
visualization	Reusable plotting, atlas, diagnostics, animation, and figure-saving helpers.

7. API Facades

Each major domain exposes a lightweight api.py facade. Use the top-level registry to discover available domains without importing heavy backends:

```
from ia_analysis import api

print(api.available_domains())
correlation_api = api.load_domain_api("correlations")
visual_api = api.load_domain_api("visualization")
```

For direct imports, prefer concise domain surfaces such as ia_analysis.shapes, ia_analysis.dynamics, ia_analysis.correlations, and ia_analysis.visualization.

8. Common Workflows

Shape measurement:

```
from ia_analysis.shapes import measure_iterative_shape, project_shape_ellipticity

shape = measure_iterative_shape(positions, masses=masses)
ellipticity = project_shape_ellipticity(shape["tensor"], los=(0.0, 0.0, 1.0))
```

Tidal-field sampling:

```
from ia_analysis.tides import build_tidal_grid, sample_tidal_grid

tidal = build_tidal_grid(particle_positions, particle_masses, boxsize=205.0)
samples = sample_tidal_grid(tidal, object_positions)
```

TNG component binding-energy profiles:

```
from ia_analysis.dynamics import compute_tng_component_binding_profiles

out = compute_tng_component_binding_profiles(
    base_path="/path/to/TNG300-1",
    snap=99,
    subhalo_id=12345,
)
summary = out["summary"]
binding_distribution = out["binding_distribution"]
```

Merger-tree cross-time workflow:

```
from ia_analysis.MergerTree import run_cross_time_workflow, save_cross_time_products

products = run_cross_time_workflow(
    base_path="/path/to/TNG300-1",
    snap0=99,
    subhalo_id0=12345,
    snap_track=[99, 91, 84, 67],
    sim_name="TNG300-1",
    components=("dm", "stars"),
)
save_cross_time_products(products, "outputs/subhalo_12345_cross_time.pkl")
```

9. Real-Space Correlations

The correlations package accepts one catalog with positions, named fields, host IDs, sample types, optional weights, and optional periodic box size.

Field	Meaning
d	Scalar density or object weight field. If absent, an all-ones field is used.
e	Shape field. It can be scalar, vector, or 3 by 3 tensor and is radially projected when needed.
v	Velocity vector field.
omega	Figure-rotation angular-velocity vector field.

Default measured statistics are ee, ed, dd, vv, dv, ev, omegae, omegad, omegav, vedomega4, and vedomega4_connected.

Categories are total, 1h, 2h, plus the five detailed classes 1h_cs, 1h_ss, 2h_cc, 2h_cs, and 2h_ss. The default suite omits 1h_cc because standard halo catalogs have at most one central per host.

```

import numpy as np
from ia_analysis.correlations import CorrelationCatalog, measure_default_correlations

catalog = CorrelationCatalog(
    positions=positions,
    fields={"e": e, "v": velocity, "omega": omega},
    host_id=host_id,
    sample_type=sample_type,
    boxsize=205.0,
)

rbins = np.logspace(-1, 1.5, 16)
suite = measure_default_correlations(catalog, rbins, covariance="jackknife", nsub=3)
ee_total = suite.results["ee"].values["total"]
ev_2h_cs_cov = suite.covariance["ev"]["2h_cs"]["cov"]

```

10. Spectra And Covariance

Power-spectrum code lives under `ia_analysis.spectra`. Mesh construction was merged into `spectra` because meshes are direct inputs to spectrum estimation. The `ia_analysis.meshes` namespace remains only for compatibility with older imports.

Velocity divergence self-folding is not treated as exact for the historical local velocity estimator because cell-wise velocity division is nonlinear. The implemented folded alternative is the additive momentum-divergence proxy `tm` in `ia_analysis.spectra.velocity_momentum`.

Spectrum covariance products live under `ia_analysis.covariance`. The module supports Gaussian, cNG, and SSC components for measured or NLA-filled spectrum products.

11. Visualization

Visualization is split by function rather than notebook name. Use `figure_io` for saving, `plot_styles` for global style, legends for shared legends/colorbars, `alignment_atlas` for exploratory alignment plots, `spectrum_plots` for $P(k)$ /AIA figures, `correlation_plots` for real-space correlation displays, `tng_dynamics_plots` for pi-closure and binding-energy diagnostics, and `merger_tree_plots` for cross-time figures.

```

from ia_analysis.visualization import set_project_style, save_figure
from ia_analysis.visualization.alignment_atlas import plot_metric_atlas

set_project_style()
fig, axes = plot_metric_atlas(frame, x_col="radius", metric_cols=["ed", "ee"],
hue_col="flag")
save_figure(fig, "alignment_metric_atlas", root="plots")

```

12. Command-Line Entrypoints

Run package-style commands after installation or with `PYTHONPATH=src`:

```

python -m ia_analysis.pipelines.run_cs --help
python -m ia_analysis.pipelines.run_tng --help
python -m ia_analysis.spectra.ia_pk_cs --help
python -m ia_analysis.spectra.ia_pk_folded --help
python -m ia_analysis.covariance.Cov --help

```

13. Notebook Workflow

Maintained notebooks live in `notebooks/pipelines`. They should orchestrate workflows and document analysis decisions. Reusable code should be moved into `src/ia_analysis` modules.

- 00_project_overview_and_paths.ipynb
- 01_global_catalog_generation.ipynb
- 02_alignment_figure_suite.ipynb
- 03_hod_population_pipeline.ipynb
- 04_power_spectrum_pipeline.ipynb
- 05_correlation_pipeline.ipynb
- 06_tng_dynamics_layered_pipeline.ipynb
- 07_orbits_and_shell_visualization.ipynb

14. Testing And Quality Checks

Run the lightweight test suite after structural changes:

```
python -m pytest tests
```

When optional HPC dependencies are missing, run smoke checks:

```
python -m compileall -q src tests
python -c "import sys; sys.path.insert(0, 'src'); import ia_analysis; print(ia_analysis.__version__)"
```

Tests should use synthetic arrays where possible. Real COSMA/TNG data checks should run separately in the data environment.

15. Development Rules

- Keep comments and docstrings in English.
- Prefer structured APIs and local helper modules over ad hoc notebook code.
- Keep low-level modules independent from pipelines.
- Add plotting helpers under `ia_analysis.visualization` by function area.
- Add tests with synthetic arrays before wiring code into real data workflows.
- Do not commit generated data products, caches, raw media outputs, or machine-specific path files.

16. Documentation Index

Document	Use
docs/architecture.md	Module boundaries and dependency direction.
docs/api_facades.md	Recommended import paths and API facades.
docs/merger_tree.md	Merger-tree and cross-time workflow steps.
docs/correlations.md	Real-space correlation categories, covariance, and four-point products.
docs/running.md	CLI entrypoints and environment variables.
docs/visualization.md	Visualization module map and notebook migration notes.
docs/velocity_divergence_self_folding.md	Velocity-divergence folding decision and tm proxy.